

Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Actividad Curricular:

IA 4.4 - Procesamiento de Imágenes I

TRABAJO PRÁCTICO N°3

1° Cuatrimestre - Año 2026

Alumnos:

LUJAMBIO, Valentín

MENDEZ, Ignacio

RAMIREZ, Lucas

Fecha:

20/06/2026

Índice

Problema – Conteo automático de cinco dados en video	3
1. Marco teórico y resolución	3
2. Desarrollo del Script e Implementación	4
2.1 Segmentación de los dados por color.....	4
2.2 Detección del reposo por seguimiento de centroides	6
2.3 Conteo de los puntos de cada cara	7
2.4 Generación del video de salida	9
Conclusión	10

Problema – Conteo automático de cinco dados en video

1. Marco teórico y resolución

El trabajo consiste en procesar videos en los que se arrojan cinco dados rojos sobre una superficie y, una vez que los dados se detienen, reconocer cada uno, contar el valor de su cara y generar un video de salida en el que cada dado aparezca recuadrado con un identificador único y su valor mientras permanezca en reposo. El problema combina dos dificultades: una temporal (decidir a partir de qué instante los dados están quietos) y otra espacial (segmentar los dados y contar sus puntos).

Los dados son rojos sobre un fondo verde uniforme, de modo que el color es el discriminador natural. La estrategia adoptada, apoyada exclusivamente en técnicas vistas en clase, se organiza en las siguientes etapas:

1. Segmentación por color en el espacio HSV (Unidad 5): se aísla el rojo de los dados con dos rangos de tono y se limpia el resultado con operaciones morfológicas de apertura y cierre (Unidad 6).
2. Detección de los dados mediante componentes conexas y filtrado por área (Unidad 7), descartando la mano y el ruido por su tamaño.
3. Determinación del reposo: se sigue el centroide de cada dado entre frames consecutivos y se considera que están quietos cuando ninguno se mueve durante una cantidad mínima de frames.
4. Conteo de los puntos (pips) de cada cara: dentro de cada dado los puntos aparecen como huecos en la máscara roja, que se cuentan como componentes conexas.
5. Generación del video de salida con el bounding box, el identificador y el valor de cada dado, visibles únicamente mientras el dado está presente.

2. Desarrollo del Script e Implementación

La solución se organiza en dos archivos. En `dados.py` reside toda la visión por computadora (segmentación, seguimiento y conteo) y en `main.py` la orquestación: recorrer los videos de la carpeta `videos/` y producir los videos clasificados. El código completo está versionado en el repositorio Git del trabajo, por lo que a continuación se prioriza la explicación del proceso y de los resultados intermedios por sobre el detalle línea por línea.

2.1 Segmentación de los dados por color

El primer paso convierte cada frame al espacio HSV, donde el color (tono) se separa de la iluminación, y se segmenta el rojo. Como el rojo se encuentra en los dos extremos del círculo de tono, se emplean dos rangos —uno cercano a 0° y otro cercano a 180° — que luego se combinan. Los umbrales mínimos de saturación y brillo descartan sombras y zonas pálidas.

```
def mascara_roja(frame_bgr, cerrar=True):
    hsv = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2HSV)
    rojo_bajo = cv2.inRange(hsv, (0, 80, 60), (10, 255, 255))
    rojo_alto = cv2.inRange(hsv, (170, 80, 60), (179, 255, 255))
    mask = cv2.bitwise_or(rojo_bajo, rojo_alto)
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k) # saca motas
    if cerrar:
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k) # rellena pips
    return mask
```

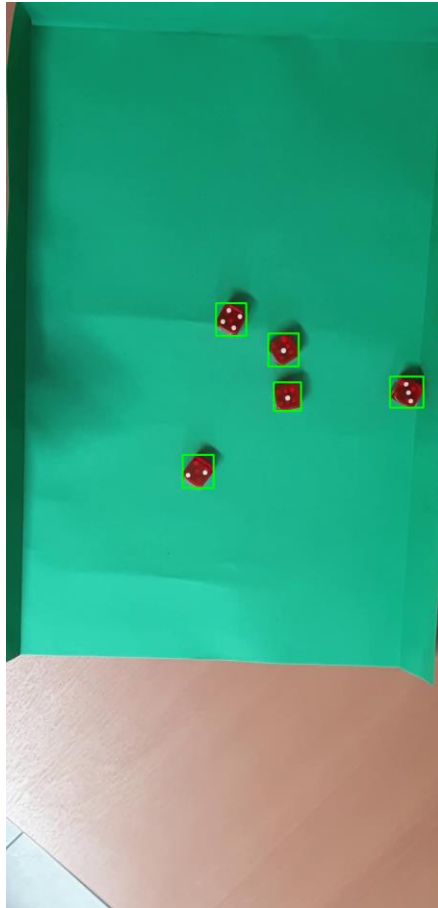
La máscara cruda obtenida solo con la umbralización de color es ruidosa: además de los dados, captura parte de la mesa de madera (cuyo tono cae cerca del rojo) y dentro de cada dado quedan agujeros en el lugar de los puntos. Para depurarla se aplica una apertura morfológica, que elimina las motas pequeñas, y opcionalmente un cierre, que rellena los

puntos y deja cada dado como un blob sólido. Se utiliza un kernel elíptico por la forma redondeada de los dados.



Tras apertura + cierre

Sobre la máscara limpia se aplica el análisis de componentes conexas. Cada componente se filtra por área para conservar únicamente los blobs del tamaño de un dado: se descartan la mano (área demasiado grande) y el ruido remanente (área demasiado chica). De cada dado válido se guarda su bounding box, su área y su centroide, que será la información que alimente las etapas siguientes.



Dados detectados tras el filtrado por área (bounding box en verde)

2.2 Detección del reposo por seguimiento de centroides

Para decidir a partir de qué frame los dados están quietos se sigue el centroide de cada dado a lo largo del video. En cada frame se emparejan los dados detectados con los del frame anterior por cercanía de centroide (el más próximo dentro de un radio máximo). Si un dado se desplazó menos que un umbral en píxeles, se incrementa su contador de "frames quieto"; si se movió más, el contador se reinicia. Esta lógica se encapsula en la clase SeguimientoCentroides.

Se considera que la tirada quedó en reposo cuando se detectan los cinco dados y los cinco acumulan al menos quince frames consecutivos sin moverse. Ese es el instante en el que se captura el frame que se usará para clasificar las caras, evitando analizar los dados mientras todavía ruedan o están parcialmente ocultos por la mano.

2.3 Conteo de los puntos de cada cara

Una vez fijado el frame en reposo, se cuenta el valor de cada dado. La idea central es que, dentro del cuerpo rojo de un dado, los puntos son zonas no rojas, es decir, huecos en la máscara de color. Por eso esta etapa reutiliza la máscara roja pero sin el cierre, de modo que los puntos se preserven como agujeros. Para cada dado se rellena el contorno de su cara y se le resta la máscara roja: la diferencia son exactamente los puntos.

```
# Apertura SIN cierre: los pips siguen siendo huecos en el rojo.
roi = mascara_roja(frame, cerrar=False)[y:y+h, x:x+w]
contornos, _ = cv2.findContours(roi, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cara_llena = np.zeros_like(roi)
cv2.drawContours(cara_llena, contornos, -1, 255, thickness=-1)
pips = cv2.subtract(cara_llena, roi) # huecos = pips
num, _, stats, _ = cv2.connectedComponentsWithStats(pips, connectivity=8)
```

```
valor = sum(1 for i in range(1, num)
            if stats[i, cv2.CC_STAT_AREA] >= pip_area_min)
```

Los puntos así obtenidos se cuentan como componentes conexas, filtrando por un área mínima para descartar pequeñas imperfecciones del borde. El número de componentes es el valor de la cara. La secuencia completa para un dado de valor cuatro se ilustra a continuación.



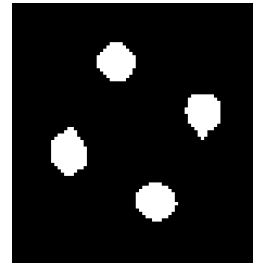
Recorte del dado



Rojo sin cerrar



Cara rellena



Puntos (resta)

Proceso de extracción de los puntos: la cara rellena menos el rojo deja los cuatro puntos

2.4 Generación del video de salida

Con el valor de cada dado y un identificador único asignado (D1 a D5), se reescribe el video rotulando cada dado. En lugar de fijar los recuadros en la posición de reposo, en cada frame se vuelven a detectar los dados y se dibuja el recuadro de un dado solo si sigue presente en su lugar. De esta forma el recuadro aparece cuando el dado se asienta y desaparece en cuanto la mano lo levanta, tal como pide la consigna. La etiqueta sigue el formato D<id>-<valor> (por ejemplo D1-4).



Dados en reposo, todos rotulados



La mano levanta un dado: su recuadro desaparece

Resultado final: los recuadros se muestran solo mientras el dado está presente

Conclusión

Apoyar toda la cadena en la segmentación de color en HSV resultó la decisión clave del trabajo: al ser los dados rojos sobre un fondo verde, un par de rangos de tono más limpia

morfológica bastan para aislarlos de forma estable, sin recurrir a técnicas fuera del programa. Sobre esa misma máscara se resolvieron tanto la detección de los dados (vía componentes conexas y filtrado por área) como el conteo de los puntos, aprovechando que los puntos son huecos en el rojo. Reutilizar la máscara con y sin cierre evitó introducir umbrales de brillo adicionales, que en las pruebas resultaban poco robustos.

El seguimiento de centroides permitió separar con claridad la fase de movimiento de la de reposo y, al re-detectar los dados en cada frame del video de salida, los recuadros aparecen y desaparecen de manera natural según los dados estén o no presentes. Como limitación, los umbrales de área de los dados y de los puntos están calibrados para la resolución de los videos provistos; ante un cambio de escala convendría expresarlos en forma relativa al tamaño del dado. Aun así, el método clasificó correctamente las cuatro tiradas de prueba.